



**Stellenbosch**

UNIVERSITY  
IYUNIVESITHI  
UNIVERSITEIT

forward together  
sonke siya phambili  
saam vorentoe

# Assignment 2

## Convex Optimization

Statistical Learning Theory  
2026

## Question 1

In this question, you are asked to write a short paper **comparing gradient descent and Newton's method** in terms of its use in statistical learning. Note that this question should be a short paper **no more than five pages** and does not require additional descriptions and/or formatting like, for example, an abstract. The following topics should be discussed in your report:

- a) **High-level overview:** Provide a **brief description of gradient descent and Newton's method** in the context of **unconstrained optimization of a loss function**. What is the **basic update rule** for each, and how do they **differ conceptually**?
- b) **Use in statistical learning:** Identify a statistical learning problem where gradient descent (or a variant of it) is commonly used. Then, identify a problem where Newton's method (or a variant of it) is more suitable. Briefly **describe any properties of the problem that make gradient descent or Newton's method more attractive**.
- c) **Convergence analysis:** Compare the two methods in terms of their convergence properties. In what types of **problems does gradient descent converge slowly**? Under what **conditions does Newton's method achieve faster convergence**? Discuss the **role of the Hessian in Newton's method and the computational trade-offs between the two methods**.
- d) **Computational complexity:** Compare gradient descent and Newton's method in terms of their computational complexity. Discuss **the cost per iteration for each** and explain **why Newton's method may become intractable for large-scale problems**. When does it **make sense to use gradient descent over Newton's method despite its slower convergence**?

**Remark:** **State any assumptions** that you make on the properties of the objective function (*e.g.* Lipschitz continuity, strong convexity). You may **limit the theoretical discussion to convex functions** and heavily rely on the section on optimization algorithms in Chapter 2. To avoid explicitly restating all these results, **rather refer to the main theorems in the book Convex Optimization, and discuss the consequences of these results**. This question is intended to help you prepare for the oral assessment and is based solely on the class notes. A concise report of 3 to 5 pages is expected, but you may exceed the page limit if you wish to explore additional results.

## Question 2

This question builds on the coordinate descent algorithm for the lasso. Refer to slides 68–76 of section 2.3 as well as the written notes 2.3.5 from page 16–18.

Consider the elastic net optimization problem:

$$\min_{\beta \in \mathbb{R}^p} R(\beta) = \frac{1}{2} \|\mathbf{y} - X\beta\|_2^2 + \lambda_1 \|\beta\|_1 + \frac{\lambda_2}{2} \|\beta\|_2^2, \quad \lambda_1 \geq 0, \lambda_2 \geq 0.$$

(a) Show that  $R(\beta)$  is convex. Under what conditions is it strongly convex?

(b) Fix all coordinates except  $\beta_j$  and define the partial residual by  $\mathbf{r}^{(-j)} = \mathbf{y} - X_{(-j)}\beta_{(-j)}$ . Derive the one-dimensional subproblem in  $\beta_j$ . Then expand the quadratic term and argue that the one-dimensional subproblem is equivalent to

$$\tilde{Q}_j(\beta_j) = \frac{1}{2}(\|\mathbf{x}_j\|_2^2 + \lambda_2)\beta_j^2 - (\mathbf{x}_j^\top \mathbf{r}^{(-j)})\beta_j + \lambda_1 |\beta_j|.$$

(c) Using subgradient optimality conditions, derive the coordinate descent update:

$$\beta_j \leftarrow \frac{S(z_j, \lambda_1)}{\|\mathbf{x}_j\|_2^2 + \lambda_2}.$$

Clearly define all quantities used.

(d) Show that the solution is sparse. Under what condition is  $\beta_j = 0$ ? Compare this to ridge regression.

(e) Do you think the elastic net will converge faster than the lasso? Hint: consider your answer in (a) and refer to slide 25 of section 2.3.

(f) Use the example code `Example_Coordinate_Descent_of_Lasso.ipynb` on EMSLearn and modify it to apply the elastic net to a simulated example. Compare your results to `scikit-learn` implementation `ElasticNet`. Note: `ElasticNet` defines the weights of the L1 and L2 norms by  $\alpha \cdot \rho$  and  $\frac{\alpha(1-\rho)}{2}$ , respectively, where `l1_ratio` =  $\rho \in [0, 1]$ . Furthermore, the squared-error loss is divided by  $n$  (sample size). Based on the definition of the elastic net in this question, you must use  $\alpha = \frac{\lambda_1 + \lambda_2}{n}$  and  $\rho = \frac{\lambda_1}{\lambda_1 + \lambda_2}$ .

- Set random states / seeds so the results are reproducible.
- Set some coefficients to zero during the simulation. See if your code correctly regularizes these directions.
- Consider adding moderate to extreme collinearity and compare the results with the lasso.

## Question 3

### Part 1

Consider the ridge regression objective

$$R(\boldsymbol{\beta}) = \frac{1}{2n} \|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \frac{\lambda}{2} \|\boldsymbol{\beta}\|_2^2, \quad \boldsymbol{\beta} \in \mathbb{R}^p,$$

where  $X \in \mathbb{R}^{n \times p}$ ,  $\mathbf{y} \in \mathbb{R}^n$ , and  $\lambda > 0$ .

- (a) Derive the exact minimizer  $\boldsymbol{\beta}^*$  of the ridge regression objective, and write it in closed form.
- (b) Compute the gradient and Hessian of  $R$ . Show that  $R$  is strongly convex, and derive its strong convexity parameter  $\tau$ .
- (c) Show that  $\nabla_{\boldsymbol{\beta}} R$  is Lipschitz continuous. Derive an expression for the Lipschitz constant  $\mu$ .

### Part 2

Refer to section 2.3, slide 24 bullets 1–2 and slide 25 bullet 2. These points state that if we solve ridge regression using gradient descent, then the algorithm will converge if  $t \leq \frac{1}{\mu}$ , where  $t$  is the step size and  $\mu$  is the Lipschitz constant. Furthermore, since the objective is strongly convex, the convergence is linear, i.e.,  $\epsilon$ -suboptimal point is obtained in  $\mathcal{O}(\log(\frac{1}{\epsilon}))$  iterations. Finally, slides 22–27 confirm that using  $\|(\nabla_{\boldsymbol{\beta}} R)(\boldsymbol{\beta})\|_2 \leq \epsilon$  as stopping condition implies that an  $\frac{\epsilon^2}{2\tau}$ -suboptimal point is obtained. You must now apply the following simulation to confirm these results. Use the following data-generating process.

- Set the random seed to 12345.
- Generate data with:
  - $n = 200$ ,
  - $p = 8$ ,
  - rows of  $X$  sampled independently from  $N(\mathbf{0}, I_p)$ ,

– true coefficient vector

$$\boldsymbol{\beta}_{\text{true}} = \begin{bmatrix} 2.0 \\ -1.5 \\ 0.0 \\ 3.0 \\ 0.0 \\ -2.5 \\ 1.0 \\ 0.5 \end{bmatrix},$$

– random noise vector  $\boldsymbol{\varepsilon} \sim N(\mathbf{0}, 0.5^2 I_n)$ ,

– response

$$\mathbf{y} = X\boldsymbol{\beta}_{\text{true}} + \boldsymbol{\varepsilon}.$$

- Take the ridge penalty parameter to be  $\lambda = 0.2$ .
- For gradient descent, use the initial value  $\boldsymbol{\beta}^{(0)} = \mathbf{0} \in \mathbb{R}^p$ .
- Use the stopping tolerance

$$\left\| (\nabla_{\boldsymbol{\beta}} R)(\boldsymbol{\beta}^{(k)}) \right\|_2 \leq 10^{-6}.$$

(d) Simulate the data. Then use the relevant **eigen decomposition** to compute the **maximum step size** to ensure convergence as well as the upper bound on the suboptimality gap under the stopping condition provided in Part 2. **Provide the analytical expressions** as well as the values based on the simulated data.

(e) For gradient descent with step size  $t = 1/\mu$ , it can be shown that the objective values satisfy the linear convergence bound

$$R(\boldsymbol{\beta}^{(k)}) - R(\boldsymbol{\beta}^*) \leq \left(1 - \frac{\tau}{\mu}\right)^k \left(R(\boldsymbol{\beta}^{(0)}) - R(\boldsymbol{\beta}^*)\right).$$

Moreover, it can be shown that, for a  $\mu$ -smooth convex function (convex with Lipschitz gradient  $\mu$ ),

$$\left\| (\nabla_{\boldsymbol{\beta}} R)(\boldsymbol{\beta}) \right\|_2^2 \leq 2\mu (R(\boldsymbol{\beta}) - R(\boldsymbol{\beta}^*)).$$

Use this information to show that a sufficient number of iterations  $k$  to guarantee that

$$\left\|(\nabla_{\beta} R)(\beta^{(k)})\right\|_2 \leq \epsilon \text{ is}$$

$$k \geq \frac{\log\left(\frac{2\mu\left(\mathcal{R}(\beta^{(0)}) - \mathcal{R}(\beta^*)\right)}{\epsilon^2}\right)}{-\log\left(1 - \frac{\tau}{\mu}\right)}.$$

(f) **Implement gradient descent** using the step size calculated in (d) and stopping tolerance  $\epsilon = 10^{-6}$ . Your code must:

- generate the data exactly as specified;
- compute  $\beta^*$  using the closed-form solution;
- compute  $\mu$  and  $\tau$  using eigen decomposition;
- run gradient descent until the stopping rule is met;
- report:
  - the final iterate  $\beta^{(k)}$ ,
  - the number of iterations,
  - the final gradient norm,
  - the observed suboptimality gap  $R(\beta^{(k)}) - R(\beta^*)$ ,
  - the theoretical upper bound  $\epsilon^2/(2\tau)$ ;
- compare the observed number of iterations with the theoretical sufficient bound.

## Bonus: Question 4

In this question, we extend gradient descent to optimization problems involving non-smooth penalty terms through proximal gradient descent.

Let  $f : \mathbb{R}^p \rightarrow \mathbb{R}$  be a differentiable function. Consider the quadratic approximation of  $f$  at a point  $\beta^-$ :

$$f(\beta) \approx f(\beta^-) + (\beta - \beta^-)^\top \nabla f(\beta^-) + \frac{1}{2t} \|\beta - \beta^-\|_2^2.$$

As seen in section 2.3, minimizing this approximation yields the update:

$$\boldsymbol{\beta}^+ = \boldsymbol{\beta}^- - t\nabla f(\boldsymbol{\beta}^-).$$

Now consider the optimization problem with a non-smooth penalty:

$$\min_{\boldsymbol{\beta} \in \mathbb{R}^p} f(\boldsymbol{\beta}) + g(\boldsymbol{\beta}),$$

where  $f$  is differentiable and  $g$  is possibly non-differentiable.

We approximate only the smooth part  $f$  and keep  $g$  exact. This results in the following optimization problem:

$$\min_{\boldsymbol{\beta}} \left\{ f(\boldsymbol{\beta}^-) + (\boldsymbol{\beta} - \boldsymbol{\beta}^-)^\top \nabla f(\boldsymbol{\beta}^-) + \frac{1}{2t} \|\boldsymbol{\beta} - \boldsymbol{\beta}^-\|_2^2 + g(\boldsymbol{\beta}) \right\},$$

which is equivalent to

$$\min_{\boldsymbol{\beta}} \left\{ (\boldsymbol{\beta} - \boldsymbol{\beta}^-)^\top \nabla f(\boldsymbol{\beta}^-) + \frac{1}{2t} \|\boldsymbol{\beta} - \boldsymbol{\beta}^-\|_2^2 + g(\boldsymbol{\beta}) \right\}.$$

By completing the square, the optimization problem can be written as

$$\min_{\boldsymbol{\beta}} \left\{ \frac{1}{2t} \|\boldsymbol{\beta} - (\boldsymbol{\beta}^- - t\nabla f(\boldsymbol{\beta}^-))\|_2^2 + g(\boldsymbol{\beta}) \right\}.$$

Define  $\mathbf{z} = \boldsymbol{\beta}^- - t\nabla f(\boldsymbol{\beta}^-)$  such that the problem can be written as

$$\boldsymbol{\beta}^+ = \arg \min_{\boldsymbol{\beta}} \left\{ \frac{1}{2t} \|\boldsymbol{\beta} - \mathbf{z}\|_2^2 + g(\boldsymbol{\beta}) \right\}.$$

This defines the proximal operator:

$$\text{prox}_{tg}(\mathbf{z}) = \arg \min_{\boldsymbol{\beta}} \left\{ \frac{1}{2t} \|\boldsymbol{\beta} - \mathbf{z}\|_2^2 + g(\boldsymbol{\beta}) \right\}.$$

Hence, the update becomes:

$$\boldsymbol{\beta}^+ = \text{prox}_{tg}(\boldsymbol{\beta}^- - t\nabla f(\boldsymbol{\beta}^-)).$$

This essentially means take the gradient descent update step, but then adjust it to respect the non-smooth penalty. Note that if we have a simple structure for  $g(\boldsymbol{\beta})$  such as

$$g(\boldsymbol{\beta}) = \sum_{j=1}^p g_j(\beta_j),$$

then the proximal operator can be solved coordinate-wise:

$$\begin{aligned} \frac{1}{2t} \|\boldsymbol{\beta} - \mathbf{z}\|_2^2 + g(\boldsymbol{\beta}) &= \sum_{j=1}^p \left[ \frac{1}{2t} (\beta_j - z_j)^2 + g_j(\beta_j) \right] \\ \implies \hat{\beta}_j &= \arg \min_{\beta_j} \left\{ \frac{1}{2t} (\beta_j - z_j)^2 + g_j(\beta_j) \right\} \text{ for } j = 1, \dots, p. \end{aligned}$$

For example, consider the L1 norm. Then we have:

$$\text{prox}_{tg}(\mathbf{z}) = \arg \min_{\boldsymbol{\beta}} \left\{ \frac{1}{2t} \|\boldsymbol{\beta} - \mathbf{z}\|_2^2 + \|\boldsymbol{\beta}\|_1 \right\}.$$

Note that this problem simplifies to coordinate-wise optimization. Furthermore, this can be interpreted (somewhat) as fitting a lasso to  $\mathbf{z}$  using the data matrix of  $I_p$ . Refer to the slides where we considered the special case of the lasso when  $\mathbf{X} = I_N$ .

Use this theory to answer the following questions.

## Problem Setup

Consider binary logistic regression with parameter vector  $\boldsymbol{\beta} \in \mathbb{R}^p$ . The binary cross-entropy loss is given by:

$$R(\boldsymbol{\beta}) = \frac{1}{N} \sum_{i=1}^N [-y_i \log(\sigma(\mathbf{x}_i^\top \boldsymbol{\beta})) - (1 - y_i) \log(1 - \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}))],$$

where  $\sigma(u) = \frac{1}{1+e^{-u}}$  is the sigmoid function. We consider the L1-regularized problem:

$$\min_{\boldsymbol{\beta}} R(\boldsymbol{\beta}) + \lambda \|\boldsymbol{\beta}\|_1.$$

(a) Write down the proximal gradient update for this problem.



(b) Provide an expression used to compute  $\mathbf{z}$ , then write down the simplified optimization with respect to a single coordinate  $\beta_j$ .

(c) **Extra Bonus:** Use the following code to simulate a binary classification problem. Then program the proximal coordinate descent algorithm for binary logistic regression with the L1 penalty. Run the algorithm and provide the estimated parameter vector, training accuracy, and test accuracy.

**Notes:**

- Do not regularize the intercept.
- Standardize the data before applying L1 penalty. This is already done in the provided code.
- Use  $\lambda = 0.08$  and a tolerance of  $\epsilon = 1e^{-07}$ .
- Note that we can specify the maximum step size before starting the procedure, since even with the L1 penalty, the eigenvalues of the Hessian are still bounded by  $\frac{1}{4N} \lambda_{\max}(\mathbf{X}^\top \mathbf{X})$ , assuming the Hessian is defined (the Hessian is not defined everywhere as we have the L1 norm).
- Therefore, use the maximum possible step size in your code, *i.e.*, use:

$$t = \frac{1}{4N} \lambda_{\max}(\mathbf{X}^\top \mathbf{X}).$$

```
import numpy as np

def sigmoid(z):
    """
    Numerically stable sigmoid.
    """
    z_clip = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z_clip))

def generate_logistic_data(
    n_samples = 1000,
    n_features = 20,
    n_informative = 5,
    seed = 42,
):
    """
    Generate binary classification data from a sparse logistic model.
    Returns
    X: (n_samples, n_features) ndarray
```

```

y: (n_samples,) ndarray in {0,1}
beta_true: (n_features,) ndarray
intercept_true: float
"""

rng = np.random.default_rng(seed)
X = rng.normal(0.0, 1.0, size=(n_samples, n_features))

beta_true = np.zeros(n_features)
beta_true[:n_informative] = np.array([2.5, -2.0, 1.5, -1.2, 0.8])[:n_informative]
intercept_true = -0.3

logits = intercept_true + X @ beta_true
probs = sigmoid(logits)
y = rng.binomial(1, probs).astype(float)

return X, y, beta_true, intercept_true

### Upper bound for step size
def lipschitz_constant_logistic(X):
    n = X.shape[0]
    spectral_norm = np.linalg.norm(X, ord=2)
    return (spectral_norm ** 2) / (4.0 * n)

# Generate data
X, y, beta_true, intercept_true = generate_logistic_data(
    n_samples=1200,
    n_features=20,
    n_informative=5,
    seed=7,
)

X_mean = X.mean(axis=0)
X_std = X.std(axis=0)
X_std[X_std == 0.0] = 1.0
Xs = (X - X_mean) / X_std

# Train/test split
n_train = 900
X_train, X_test = Xs[:n_train], Xs[n_train:]
y_train, y_test = y[:n_train], y[n_train:]

```